25VL684 FUNCTIONAL VERIFICATION LAB

VERIFICATION OF A TRAFFIC LIGHT CONTROLLER USING SYSTEM VERILOG LAYERED TESTBENCH ARCHITECTURE

Name : Vedasri J
Roll No : CB.EN.P2VLD25035
Date : 30/03/2026

1. AIM:

To verify a Traffic Light Controller (TLC) using a SystemVerilog layered testbench architecture. The verification environment includes a generator, driver, monitor, scoreboard, and functional coverage collector, with embedded SVA properties for formal and simulation-based checking.

2. THEORY:

A TLC is a digital hardware module that manages signal heads at a road intersection. The controller implements a Finite State Machine (FSM) that cycles through defined light phases — green, yellow, and red — for each traffic direction. The FSM Controller controls signal heads in both North-South (NS) and East-West (EW) directions.

The TLC designed here extends the baseline FSM with 2 key features:

1. Emergency Vehicle Preemption — asserts all-red immediately when emg_req is raised, preventing conflicts during emergency vehicle passage.
2. Fault Detection — monitors for conflicting outputs and stuck-state conditions, transitioning to a safe all-red state on any anomaly.

Verification is implemented as a layered SystemVerilog testbench. Stimulus generation, signal driving, output monitoring, and result checking are separated into independent reusable classes. Randomised transaction objects flow through mailboxes (gen2drv, mon2scb) and a virtual interface connects the class-based environment to the DUT signals.

3. BLOCK DIAGRAM:

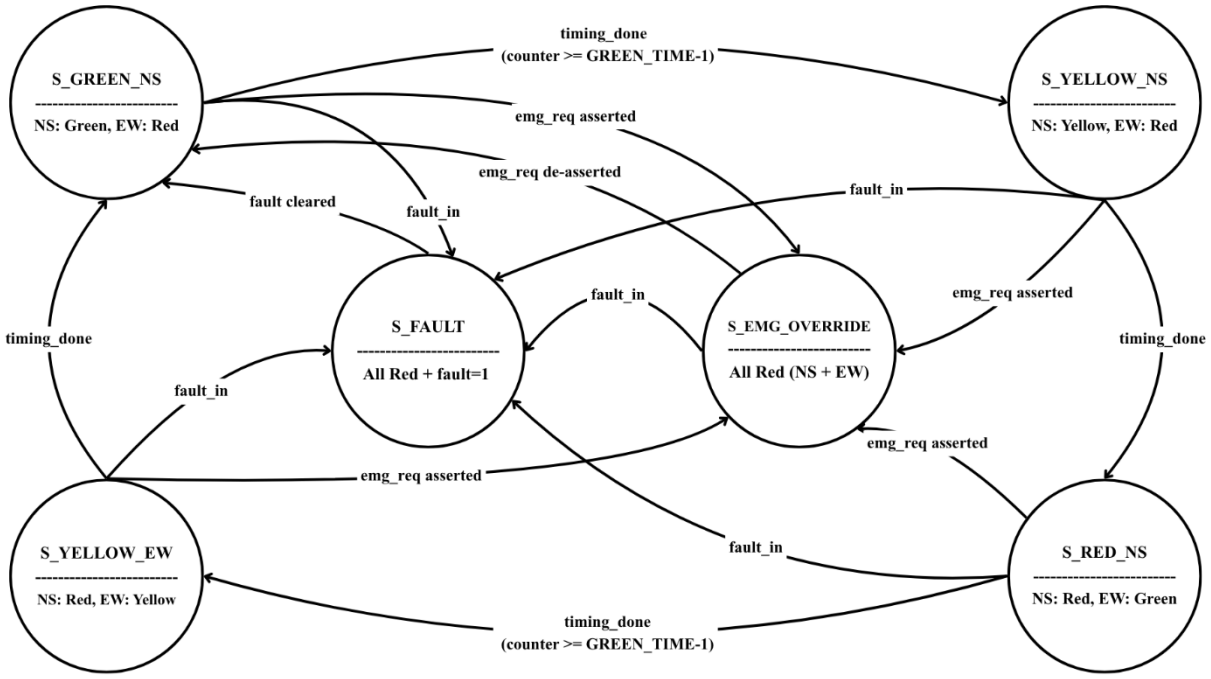
The block diagram illustrates the top-level architecture of the Traffic Light Controller. The FSM Controller is the central block, receiving input signals and driving output signal heads. The FSM Controller receives clk, rst_n, sensor[1:0], emg_req, and fault_in, and produces light_NS[2:0], light_EW[2:0], fault_out, and state_dbg[2:0]. State transitions are governed by an internal phase counter and a priority arbitration chain (fault > emg > normal timing).

Port Specification:

Port Name	Direction	Width	Description
clk	Input	1-bit	System clock — rising edge active
rst_n	Input	1-bit	Active-low asynchronous reset
sensor[1:0]	Input	2-bit	Vehicle presence: {NS, EW}

emg_req	Input	1-bit	Emergency vehicle preemption request (level-sensitive)
fault_in	Input	1-bit	External fault / sensor failure signal
light_NS[2:0]	Output	3-bit	NS signal head: {Red, Yellow, Green}
light_EW[2:0]	Output	3-bit	EW signal head: {Red, Yellow, Green}
fault_out	Output	1-bit	System fault indicator
state_dbg[2:0]	Output	3-bit	Current FSM state for debug / coverage

4. FSM STATE DIAGRAM:



The controller operates as a six-state Moore FSM. The four normal-flow states form a clockwise ring. Two exception states — S_EMG_OVERRIDE and S_FAULT — are reachable from any state and each has a defined recovery path back to S_GREEN_NS.

State Transition Table

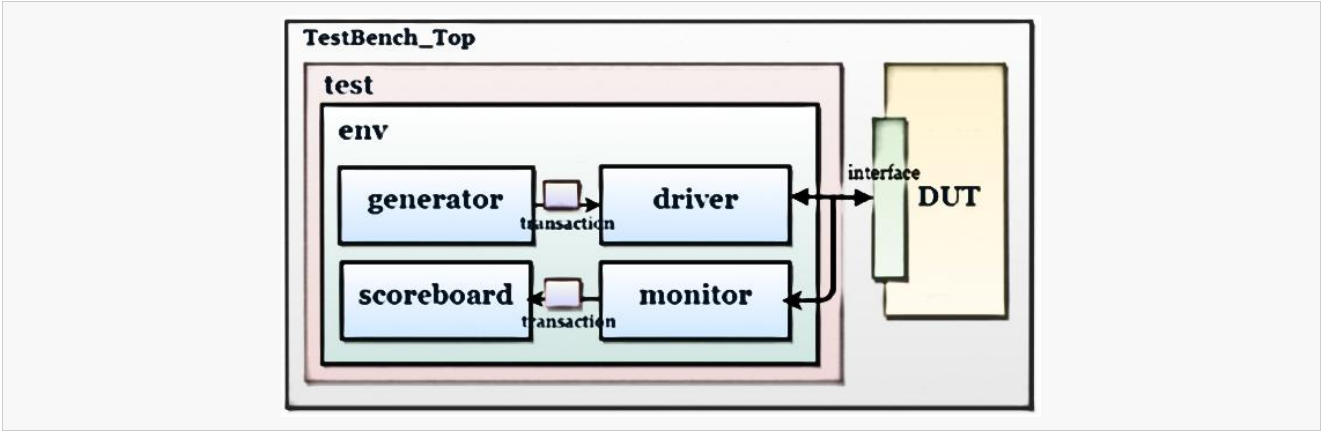
State	NS / EW Lights	Transition Condition	Next State
S_GREEN_NS	NS=Green, EW=Red	timing_done (counter >= GREEN_TIME−1)	S_YELLOW_NS
S_YELLOW_NS	NS=Yellow, EW=Red	timing_done (counter >= YELLOW_TIME−1)	S_RED_NS
S_RED_NS	NS=Red, EW=Green	timing_done (counter >= GREEN_TIME−1)	S_YELLOW_EW
S_YELLOW_EW	NS=Red, EW=Yellow	timing_done (counter >= YELLOW_TIME−1)	S_GREEN_NS
S_EMG_OVERRIDE	All Red (NS + EW)	emg_req de-asserted	S_GREEN_NS
S_FAULT	All Red + fault_out=1	fault_in de-asserted	S_GREEN_NS

5. LAYERED TESTBENCH ARCHITECTURE:

The verification environment separates concerns into five independent layers. Communication between layers uses SystemVerilog mailboxes, and a virtual interface handle connects the class-based components to the DUT signals.

5.1 ENVIRONMENT HIERARCHY

TestBench_Top instantiates the test, which builds the env. The env creates mailboxes gen2drv and mon2scb, then instantiates the generator, driver, monitor, and scoreboard, passing the virtual interface handle to the driver and monitor. Functional coverage is handled by the covergroup inside tlc_if.



5.2 COMPONENT DESCRIPTIONS:

Component	Responsibility
Transaction	Encapsulates all stimulus and observed-output fields. Constrained-random variables cover sensor patterns, emg injection timing, duration, and fault probability (40%).
Generator	Randomises 200 transaction objects and puts them into gen2drv mailbox. Every 5th transaction forces inject_emg=1 and every 7th forces inject_fault=1 to guarantee coverage bin closure. Fires a done event on completion.
Driver	Bus Functional Model. Reads transactions from gen2drv, drives rst_n, sensor, emg_req, and fault_in cycle-accurately. Waits for the NS phase to complete before asserting EMG so the EW phase is reachable.
Monitor	Passive observer. Samples light_NS, light_EW, fault_out, and state_dbg on every posedge clk and forwards a transaction to mon2scb.
Scoreboard	Checks every observed transaction: (1) no conflicting green, (2) valid light encoding, (3) fault_out=1 implies all-red. Reports total PASS/FAIL at end.

5.2.1 TRANSACTION:

The Transaction class is the fundamental unit of communication in the testbench. It encapsulates all the signals required to stimulate the TLC and capture its response. For the TLC DUT, the transaction contains the following fields:

- **emg_inj_cycle** – cycle offset within the active phase at which emg_req is asserted.
- **emg_duration** – number of cycles for which emg_req is held high.
- **sensor[1:0]** – vehicle presence pattern for both NS and EW directions.
- **green_time** – randomised green phase duration within [MIN_GREEN : MAX_GREEN].
- **inject_fault** – flag that triggers fault injection during the transaction.
- inject_emg — flag that triggers emergency request injection during the transaction.

- obs_light_NS / obs_light_EW / obs_fault_out / obs_state — observed DUT output values, filled by the monitor and forwarded to the scoreboard.

Input fields are randomised to generate diverse test scenarios. The same transaction object type is used by the monitor to capture DUT output values into the obs_ fields for scoreboard comparison.

5.2.2 GENERATOR:

The Generator is responsible for creating stimulus for the TLC. It randomises transaction objects and sends them to the driver through the gen2drv mailbox. The generator performs the following functions:

- Randomises transaction fields such as inject_emg, emg_inj_cycle, sensor, green_time, and inject_fault.
- Creates diverse stimulus sequences covering normal phase cycling, emergency preemption, and fault injection.
- Sends each generated transaction to the driver using the gen2drv mailbox.
- Controls the total number of transactions executed per test run via a configurable parameter.

This randomised stimulus helps verify all six FSM states, all transition conditions, and the priority arbitration chain under a wide range of input combinations.

5.2.3 DRIVER:

The Driver acts as a Bus Functional Model (BFM) that converts transaction-level information into signal-level activity on the DUT interface. It receives transaction objects from the generator through the gen2drv mailbox and drives the corresponding signals on the TLC interface. The driver performs the following tasks:

- Reads transactions from the gen2drv mailbox.
- Drives DUT input signals: clk, rst_n, sensor[1:0], emg_req, and fault_in.
- Asserts emg_req at emg_inj_cycle within the active phase and holds it for emg_duration cycles.
- Asserts fault_in immediately when inject_fault is set, holds for 3 cycles, then de-asserts and allows 5 cycles for FSM recovery.
- Synchronises all signal activity with the rising edge of clk.
- Performs reset initialisation using rst_n at the start of each test.

Through this mechanism, the driver applies precise, cycle-accurate stimulus to the TLC design including the corner cases demanded by emergency preemption and fault injection.

5.2.4 MONITOR

The Monitor observes the DUT output signals and converts signal-level activity back into transaction-level information. It continuously samples the interface on every clock cycle and performs the following operations:

- Samples light_NS[2:0] and light_EW[2:0] on every posedge clk.
- Captures fault_out and state_dbg[2:0] to detect fault state entry and recovery.
- Packages the captured output values into a transaction object.
- Sends the observed transaction to the scoreboard through the mon2scb mailbox.

The monitor is a passive component and does not drive any signals to the DUT. It operates concurrently with the driver throughout the simulation.

5.2.5 SCOREBOARD

The Scoreboard is responsible for verifying the correctness of the TLC behaviour. It compares the actual DUT output against a reference FSM model for every transaction received from the monitor. The scoreboard performs the following checks:

- No conflicting green: verifies that light_NS[0] and light_EW[0] are never simultaneously asserted.
- Valid encoding: verifies that light_NS and light_EW always carry exactly one of 3'b001, 3'b010, or 3'b100.

- Fault response: verifies that fault_out=1 implies both signal heads are red.

If the observed output matches the reference model, the transaction is marked PASS; otherwise an error message is printed with the expected and actual values.

5.2.6 COVERAGE COLLECTOR

The coverage collector is implemented as a single covergroup cg instantiated inside the interface. It contains four coverpoints and two cross coverpoints:

- state_cp — all six FSM states (bins: green_ns, yellow_ns, red_ns, yellow_ew, emg_override, fault_state)
- sensor_cp — all four sensor[1:0] combinations (s00, s01, s10, s11)
- emg_cp — emg_req asserted (emg_asserted)
- fault_cp — fault_in asserted (fault_asserted)
- emg_in_state — cross of state_cp × emg_cp
- fault_in_state — cross of state_cp × fault_cp

5.2.7 ENVIRONMENT

The Environment class acts as a container that instantiates and connects all verification components. It performs the following tasks:

- Instantiates the Generator, Driver, Monitor, and Scoreboard. Functional coverage is collected automatically by the covergroup cg inside tlc_if.
- Creates mailboxes gen2drv and mon2scb for inter-component communication.
- Passes the virtual interface handle to the Driver and Monitor.
- Controls the overall execution flow: calls drv.reset(), forks all components with join_none, waits on gen.done event, drains the pipeline for 200,000 ns, then calls scb.report() and prints coverage.

5.2.8 TEST

A single test sequence is run via the env class. The generator produces 200 constrained-random transactions, with every 5th transaction forcing inject_emg=1 and every 7th forcing inject_fault=1 to guarantee coverage bin closure. No separate test class hierarchy is used.

5.2.9 TOP-LEVEL TESTBENCH

The top-level testbench module (top_tb) connects the DUT with the verification environment. It performs the following functions:

- Instantiates the TLC DUT (traffic_light_top).
- Instantiates the interface (tlc_if) containing all DUT signals.
- Generates the clock signal clk with the required period.
- Generates the initial reset sequence on rst_n.
- Passes the virtual interface handle to the environment using a virtual interface.

The interface acts as a bridge between the DUT signals and the object-oriented testbench components, keeping the RTL-level signal connections separate from the class-based layers.

6. SYSTEMVERILOG ASSERTIONS (SVA):

Concurrent SVA properties are embedded inside the interface and checked throughout simulation. Three safety properties are defined and checked throughout simulation.

Safety Properties

```
// P1 — No conflicting green (both directions must never be green together)
    property no_conflicting_green;
        @(posedge clk) disable iff (!rst_n)
        !(light_NS[0] && light_EW[0]);
    endproperty
    P1_no_conflict: assert property (no_conflicting_green)
        else $error("[SVA FAIL P1] @ %0t ns — Conflicting GREEN: NS=%03b EW=%03b",
                    $time, light_NS, light_EW);

    // P2 — Emergency: all red within 1 clock cycle
    property emg_all_red;
        @(posedge clk) disable iff (!rst_n)
        emg_req |=> (light_NS[2] && light_EW[2]);
    endproperty
    P2_emg_allred: assert property (emg_all_red)
        else $error("[SVA FAIL P2] @ %0t ns — EMG did not force ALL-RED", $time);

    // P3 — Fault output: both signals must be red
    property fault_all_red;
        @(posedge clk) disable iff (!rst_n)
        fault_out |-> (light_NS[2] && light_EW[2]);
    endproperty
    P3_fault_allred: assert property (fault_all_red)
        else $error("[SVA FAIL P3] @ %0t ns — FAULT active but not ALL-RED", $time);
```

7. RANDOMISATION CONSTRAINTS:

Constraint	Effect
sensor_valid	sensor inside {2'b00, 2'b01, 2'b10, 2'b11} — all four combinations valid
timing_valid	green_time inside {[4:10]} — bounded phase duration
emg_dur_valid	emg_duration inside {[1:5]} — hold emg_req for 1–5 cycles
emg_dist_c	inject_emg dist {0:=50, 1:=50} — EMG injected in 50% of transactions
fault_dist_c	inject_fault dist {0:=60, 1:=40} — fault injected in 40% of transactions
no_simultaneous	!(inject_emg && inject_fault) — orthogonal stimulus; never simultaneous
emg_cycle_var	if (inject_emg) emg_inj_cycle inside {[1:8]} — late injection reaches EW phase

8. CODES:

8.1 tlc_if.sv

```
//=======================================
// INTERFACE : tlc_if
//=======================================
interface tlc_if (input logic clk);


    logic        rst_n;
    logic [1:0]  sensor;
    logic        emg_req;
    logic        fault_in;
    logic [2:0]  light_NS;
    logic [2:0]  light_EW;
    logic        fault_out;
    logic [2:0]  state_dbg;


    // ------------------------------------
    // SVA : Safety Properties
    // ------------------------------------


    // P1 — No conflicting green (both
directions must never be green together)
    property no_conflicting_green;
        @(posedge clk) disable iff
(!rst_n)
        !(light_NS[0] && light_EW[0]);
    endproperty
    P1_no_conflict: assert property
(no_conflicting_green)
        else $error("[SVA FAIL P1] @ %0t
ns — Conflicting GREEN: NS=%03b EW=%03b",
                    $time, light_NS,
light_EW);


    // P2 — Emergency: all red within 1
clock cycle
    property emg_all_red;
        @(posedge clk) disable iff
(!rst_n)
        emg_req |=> (light_NS[2] &&
light_EW[2]);
    endproperty
    P2_emg_allred: assert property
(emg_all_red)
        else $error("[SVA FAIL P2] @ %0t
ns — EMG did not force ALL-RED", $time);


    // P3 — Fault output: both signals
must be red
    property fault_all_red;
        @(posedge clk) disable iff
(!rst_n)
        fault_out |-> (light_NS[2] &&
light_EW[2]);
    endproperty
    P3_fault_allred: assert property
(fault_all_red)
        else $error("[SVA FAIL P3] @ %0t
ns — FAULT active but not ALL-RED",
$time);


    // ------------------------------------
    // Functional Coverage
    // ------------------------------------
    covergroup cg @(posedge clk);


        // All six FSM states
        state_cp: coverpoint state_dbg {
            bins green_ns      = {3'd0};
            bins yellow_ns     = {3'd1};
            bins red_ns        = {3'd2};
            bins yellow_ew     = {3'd3};
            bins emg_override  = {3'd4};
            bins fault_state   = {3'd5};
        }


        // All four sensor combinations
        sensor_cp: coverpoint sensor {
            bins s00 = {2'b00};
            bins s01 = {2'b01};
            bins s10 = {2'b10};
            bins s11 = {2'b11};
```

```
        }

        // Emergency triggered
        emg_cp: coverpoint emg_req {
            bins emg_asserted = {1'b1};
        }

        // Fault triggered
        fault_cp: coverpoint fault_in {
            bins fault_asserted = {1'b1};
        }

        // Cross: which state was active when emergency arrived
        emg_in_state: cross state_cp, emg_cp;

        // Cross: which state was active when fault arrived
        fault_in_state: cross state_cp, fault_cp;

    endgroup

    cg cg_inst = new();

endinterface
```

8.2 transaction.sv

```
//=========================================
// TRANSACTION
//
=========================================
class transaction;

    // Stimulus fields
    rand bit [1:0]  sensor;
    rand int        emg_inj_cycle;
    rand int        emg_duration;
    rand int        green_time;
    rand bit        inject_fault;
    rand bit        inject_emg;

    // Observed outputs (filled by monitor)
    bit [2:0]  obs_light_NS;
    bit [2:0]  obs_light_EW;
    bit        obs_fault_out;
    bit [2:0]  obs_state;

    // ---- Constraints ----
    constraint sensor_valid    { sensor inside {2'b00, 2'b01, 2'b10, 2'b11}; }
    constraint timing_valid    { green_time inside {[4:10]}; }
    constraint emg_cycle_valid { emg_inj_cycle inside {[1:4]}; }
    constraint emg_dur_valid   { emg_duration  inside {[1:5]}; }

    // Fault injected 40% of the time; never simultaneously with emg
    constraint fault_dist_c { inject_fault dist {0:=60, 1:=40}; }

constraint emg_dist_c {
    inject_emg dist {0:=50, 1:=50};
}
    constraint no_simultaneous { !(inject_fault && inject_emg); }

constraint emg_cycle_var {
    if (inject_emg) emg_inj_cycle inside {[1:8]};
}

    function void display(string tag);
```

```
        $display("[%s] sensor=%02b
inject_emg=%0b emg_inj=%0d emg_dur=%0d
inject_fault=%0b",

                 tag, sensor, inject_emg,
emg_inj_cycle, emg_duration,
inject_fault);

    endfunction


endclass
```

8.3 generator.sv

```
//=========================================

// GENERATOR

//=========================================

class generator;


    mailbox  gen2drv;

    int      num_trans = 200;

    event    done;


    function new(mailbox gen2drv);

        this.gen2drv = gen2drv;

    endfunction


    task run();

        transaction tr;


        for (int i = 0; i < num_trans;
i++) begin


            tr = new();


            if (!tr.randomize())

                $fatal(1, "[GEN]
Randomization failed");


            // FORCE EVENTS FOR COVERAGE
CLOSURE

            if (i % 5 == 0) begin

                tr.inject_emg = 1;

                tr.inject_fault = 0;

            end


            if (i % 7 == 0) begin

                tr.inject_fault = 1;

                tr.inject_emg   = 0;

            end


            tr.display($sformatf("GEN-
T%0d", i));


            gen2drv.put(tr);

        end


        $display("[GEN] Done — %0d
transactions sent.", num_trans);


        -> done;   // signal completion

    endtask


endclass
```

8.4 driver.sv

```
//=========================================

// DRIVER

//=========================================

class driver;


    virtual tlc_if  vif;

    mailbox         gen2drv;


    function new(virtual tlc_if vif,
mailbox gen2drv);
```

```
        this.vif     = vif;
        this.gen2drv = gen2drv;
    endfunction

    // reset task - called explicitly
before fork in env.run()
    task reset();
        vif.rst_n    <= 1'b0;
        vif.sensor   <= 2'b00;
        vif.emg_req  <= 1'b0;
        vif.fault_in <= 1'b0;
        repeat (3) @(posedge vif.clk);
        vif.rst_n <= 1'b1;
        @(posedge vif.clk);
        $display("[DRV] Reset complete.");
    endtask

    task run();
        transaction tr;
        forever begin
            gen2drv.get(tr);
            tr.display("DRV");
            drive_one(tr);
        end
    endtask

    task drive_one(transaction tr);
        // Apply sensor
        vif.sensor <= tr.sensor;

        // Normal phase cycles
        repeat (tr.green_time) @(posedge
vif.clk);

        // Emergency injection
        if (tr.inject_emg) begin
            repeat (tr.emg_inj_cycle)
@(posedge vif.clk);
            vif.emg_req <= 1'b1;
            $display("[DRV] emg_req
asserted for %0d cycles",
tr.emg_duration);
            repeat (tr.emg_duration)
@(posedge vif.clk);
            vif.emg_req <= 1'b0;
            @(posedge vif.clk);
        end

        // Fault injection
        if (tr.inject_fault) begin
            vif.fault_in <= 1'b1;
            $display("[DRV] fault_in
asserted");
            repeat (3) @(posedge vif.clk);
            vif.fault_in <= 1'b0;
            $display("[DRV] fault_in de-
asserted");
            repeat (5) @(posedge vif.clk);
// allow recovery
        end

        // Gap between transactions
        vif.sensor <= 2'b00;
        repeat (2) @(posedge vif.clk);
    endtask

endclass
```

8.5 monitor.sv

```
//=========================================
// MONITOR
//=========================================
class monitor;

    virtual tlc_if  vif;
```

```
    mailbox         mon2scb;


    function new(virtual tlc_if vif,
mailbox mon2scb);
        this.vif     = vif;
        this.mon2scb = mon2scb;
    endfunction


    task run();
        transaction tr;
        forever begin
            @(posedge vif.clk);
            tr = new();
            // Store observed DUT outputs
into transaction fields
            tr.obs_light_NS  =
vif.light_NS;
            tr.obs_light_EW  =
vif.light_EW;
            tr.obs_fault_out =
vif.fault_out;
            tr.obs_state     =
vif.state_dbg;
            mon2scb.put(tr);
        end
    endtask


endclass
```

8.6 scoreboard.sv

```
//=========================================
// SCOREBOARD
//=========================================
class scoreboard;


    mailbox mon2scb;
    int     pass_count = 0;
    int     fail_count = 0;


    function new(mailbox mon2scb);
        this.mon2scb = mon2scb;
    endfunction


    task run();
        transaction tr;
        forever begin
            mon2scb.get(tr);
            check(tr);
        end
    endtask


    function void check(transaction tr);
        bit ok = 1;
        // Check 1: No conflicting green
        if (tr.obs_light_NS[0] &&
tr.obs_light_EW[0]) begin
            $display("[SCB FAIL] @ %0t -
Conflicting GREEN: NS=%03b EW=%03b",
                     $time,
tr.obs_light_NS, tr.obs_light_EW);
            ok = 0;
        end


        // Check 2: Valid light encoding
(only one bit may be set)
        if (!(tr.obs_light_NS inside
{3'b001, 3'b010, 3'b100})) begin
            $display("[SCB FAIL] @ %0t -
Invalid NS encoding: %03b",
                     $time,
tr.obs_light_NS);
            ok = 0;
        end
        if (!(tr.obs_light_EW inside
{3'b001, 3'b010, 3'b100})) begin
            $display("[SCB FAIL] @ %0t -
Invalid EW encoding: %03b",
                     $time,
tr.obs_light_EW);
            ok = 0;
        end
```

```
        // Check 3: fault_out=1 implies
both must be red

        if (tr.obs_fault_out &&
!(tr.obs_light_NS[2] &&
tr.obs_light_EW[2])) begin

            $display("[SCB FAIL] @ %0t –
fault_out=1 but not ALL-RED: NS=%03b
EW=%03b",

                        $time,
tr.obs_light_NS, tr.obs_light_EW);

            ok = 0;

        end



        if (ok) begin

            pass_count++;

            $display("[SCB PASS] @ %0t
state=%0d NS=%03b EW=%03b fault=%0b",

                        $time, tr.obs_state,
tr.obs_light_NS, tr.obs_light_EW,
tr.obs_fault_out);

        end else begin

            fail_count++;

        end

    endfunction



    function void report();


$display("================================
==========");

        $display("[SCB] TOTAL PASS = %0d",
pass_count);

        $display("[SCB] TOTAL FAIL = %0d",
fail_count);

        if (fail_count == 0)

            $display("[SCB] ALL CHECKS
PASSED");

        else

            $display("[SCB] %0d FAILURE(S)
DETECTED", fail_count);


$display("================================
==========");

    endfunction


endclass
```

8.7 env.sv

```
//========================================

// ENVIRONMENT

//========================================

class env;


    generator   gen;

    driver      drv;

    monitor     mon;

    scoreboard  scb;


    mailbox gen2drv;

    mailbox mon2scb;


    virtual tlc_if vif;


    function new(virtual tlc_if vif);

        this.vif = vif;

        gen2drv  = new();

        mon2scb  = new();

        gen      = new(gen2drv);

        drv      = new(vif, gen2drv);

        mon      = new(vif, mon2scb);

        scb      = new(mon2scb);

    endfunction


    task run();

        // Reset DUT before launching
stimulus

        drv.reset();


        // Fork all components

        fork

            gen.run();

            drv.run();
```

```
            mon.run();
            scb.run();
        join_none

        // Wait for generator to finish, then drain the pipeline
        wait (gen.done.triggered);
        #200000;
        scb.report();
        $display("[ENV] Functional coverage: %.1f%%",
vif.cg_inst.get_coverage());
    endtask

endclass
```

8.8 traffic_light_top.sv

```
//=======================================
// DUT : traffic_light_top
//=======================================
module traffic_light_top (
    input  logic        clk,
    input  logic        rst_n,
    input  logic [1:0]  sensor,
    input  logic        emg_req,
    input  logic        fault_in,
    output logic [2:0]  light_NS,   // {Red, Yellow, Green}
    output logic [2:0]  light_EW,
    output logic        fault_out,
    output logic [2:0]  state_dbg
);

    typedef enum logic [2:0] {
        S_GREEN_NS     = 3'd0,
        S_YELLOW_NS    = 3'd1,
        S_RED_NS       = 3'd2,
        S_YELLOW_EW    = 3'd3,
        S_EMG_OVERRIDE = 3'd4,
        S_FAULT        = 3'd5
    } state_t;

    // ---- Timing parameters ----
    localparam int GREEN_TIME  = 6;   // cycles in green  phase
    localparam int YELLOW_TIME = 3;   // cycles in yellow phase

    state_t state, next_state;
    int     counter;                  // phase-local cycle counter

    // ---- State register ----
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            state <= S_GREEN_NS;
        else
            state <= next_state;
    end

    // ---- Phase counter - resets on every state change ----
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            counter <= 0;
        else if (state != next_state)   // reset counter on state change
            counter <= 0;
        else
            counter <= counter + 1;
    end

    // ---- Next-state logic (priority: fault > emg > normal timing) ----
```

```
    always_comb begin

        next_state = state;


        // Highest priority: fault (hold
until fault_in clears)

        if (fault_in)

            next_state = S_FAULT;


        // Second priority: emergency
override

        else if (emg_req)

            next_state = S_EMG_OVERRIDE;


        // Normal flow

        else begin

            case (state)

                S_GREEN_NS    : if
(counter >= GREEN_TIME  - 1) next_state =
S_YELLOW_NS;

                S_YELLOW_NS   : if
(counter >= YELLOW_TIME - 1) next_state =
S_RED_NS;

                S_RED_NS      : if
(counter >= GREEN_TIME  - 1) next_state =
S_YELLOW_EW;

                S_YELLOW_EW   : if
(counter >= YELLOW_TIME - 1) next_state =
S_GREEN_NS;

                S_EMG_OVERRIDE: if
(!emg_req)                   next_state =
S_GREEN_NS;

                S_FAULT       : if
(!fault_in)                  next_state =
S_GREEN_NS;

                default       :
next_state = S_GREEN_NS;

            endcase

        end

    end


    // ---- Output logic (Moore) ----

    always_comb begin

        // Safe defaults

        light_NS  = 3'b100;  // Red

        light_EW  = 3'b100;  // Red

        fault_out = 1'b0;


        case (state)

            S_GREEN_NS    : begin light_NS
= 3'b001; light_EW = 3'b100; end  // NS
green

            S_YELLOW_NS   : begin light_NS
= 3'b010; light_EW = 3'b100; end  // NS
yellow

            S_RED_NS      : begin light_NS
= 3'b100; light_EW = 3'b001; end  // EW
green

            S_YELLOW_EW   : begin light_NS
= 3'b100; light_EW = 3'b010; end  // EW
yellow

            S_EMG_OVERRIDE: begin light_NS
= 3'b100; light_EW = 3'b100; end  // all
red

            S_FAULT       : begin light_NS
= 3'b100; light_EW = 3'b100;


fault_out = 1'b1; end
// all red + fault

            default       : begin light_NS
= 3'b100; light_EW = 3'b100; end

        endcase

    end


    assign state_dbg = state;


endmodule
```

---

9. WORKING OF THE VERIFICATION SYSTEM:

At the start of simulation, the driver applies reset to initialise the DUT into the S_GREEN_NS state. The generator then produces constrained-random transaction objects, which are passed to the driver through the gen2drv mailbox.

The driver converts each transaction into cycle-accurate stimulus by driving sensor, emg_req, and fault_in signals on the interface. Based on these inputs, the DUT FSM transitions between states and generates corresponding outputs light_NS, light_EW, and fault_out.

The monitor passively samples these outputs on every clock edge and packages them into transaction objects, which are sent to the scoreboard via the mon2scb mailbox. The scoreboard checks each transaction for correctness, ensuring no conflicting green signals, valid encoding, and correct fault behavior.

Simultaneously, SystemVerilog Assertions (SVA) embedded in the interface continuously verify safety properties such as no conflicting green, emergency all-red response, and fault-induced safe state. Any violation is immediately reported during simulation.

Functional coverage is collected in parallel through covergroups defined in the interface. Coverage points track FSM state transitions, sensor combinations, emergency events, and fault events, while cross coverage ensures that these events are exercised across all relevant states.

Thus, the combined operation of constrained-random stimulus, assertions, scoreboard checking, and coverage collection ensures comprehensive functional verification of the Traffic Light Controller.

10. SIMULATION RESULTS:

10.1 SCOREBOARD SUMMARY:

```
[GEN-T197] sensor=11 inject_emg=1 emg_inj=1 emg_dur=1 inject_fault=0
[GEN-T198] sensor=00 inject_emg=1 emg_inj=1 emg_dur=5 inject_fault=0
[GEN-T199] sensor=00 inject_emg=0 emg_inj=2 emg_dur=2 inject_fault=0
[GEN] Done — 200 transactions sent.
[DRV] sensor=01 inject_emg=0 emg_inj=3 emg_dur=3 inject_fault=1
[SCB PASS] @ 45000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 55000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 65000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 75000 state=0 NS=001 EW=100 fault=0
[DRV] fault_in asserted
[SCB PASS] @ 85000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 95000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 105000 state=5 NS=100 EW=100 fault=1
[DRV] fault_in de-asserted
[SCB PASS] @ 115000 state=5 NS=100 EW=100 fault=1
[SCB PASS] @ 125000 state=5 NS=100 EW=100 fault=1
[SCB PASS] @ 135000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 145000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 155000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 165000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 175000 state=0 NS=001 EW=100 fault=0
[DRV] sensor=10 inject_emg=1 emg_inj=4 emg_dur=1 inject_fault=0
[SCB PASS] @ 185000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 195000 state=1 NS=010 EW=100 fault=0
```

...

```
[SCB PASS] @ 205000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 215000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 225000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 235000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 245000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 255000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 265000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 275000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 285000 state=3 NS=100 EW=010 fault=0
[SCB PASS] @ 295000 state=3 NS=100 EW=010 fault=0
[DRV] emg_req asserted for 1 cycles
[SCB PASS] @ 305000 state=3 NS=100 EW=010 fault=0
[SCB PASS] @ 315000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 325000 state=4 NS=100 EW=100 fault=0
[SCB PASS] @ 335000 state=0 NS=001 EW=100 fault=0
[DRV] sensor=01 inject_emg=1 emg_inj=2 emg_dur=5 inject_fault=0
[SCB PASS] @ 345000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 355000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 365000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 375000 state=0 NS=001 EW=100 fault=0
[SCB PASS] @ 385000 state=0 NS=001 EW=100 fault=0
```

...

```
[SCB PASS] @ 945000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 955000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 965000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 975000 state=2 NS=100 EW=001 fault=0
[DRV] emg_req asserted for 4 cycles
[SCB PASS] @ 985000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 995000 state=2 NS=100 EW=001 fault=0
INFO: [USF-XSim-96] XSim completed. Design snapshot 'top_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:12 ; elapsed = 00:00:16 . Memory (MB): peak = 1019.219 ; gain = 0.000
run all
```

....

```
[SCB PASS] @ 199995000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 200005000 state=1 NS=010 EW=100 fault=0
[SCB PASS] @ 200015000 state=2 NS=100 EW=001 fault=0
[SCB PASS] @ 200025000 state=2 NS=100 EW=001 fault=0
==========================================
[SCB] TOTAL PASS = 19999
[SCB] TOTAL FAIL = 0
[SCB] ALL CHECKS PASSED ?
==========================================
[ENV] Functional coverage: 94.4%
[TOP] Simulation complete at 200035000 ns
$finish called at time : 200035 ns : File "C:/Users/23ved/vivado experiments/S2/FV_TrafficLightController,
```

Metric	Value
Total Clock Cycles Checked	19,999
PASS	19,999
FAIL	0
Functional Coverage	94.4%
Simulation End Time	200,035 ns
Simulator	Vivado XSim 2020.1

10.2 SVA ASSERTION RESULTS:

ID	Property	Result
P1	No conflicting green (!(light_NS[0] && light_EW[0]))	PASS — never violated
P2	Emergency all-red (emg_req |=> all-red next cycle)	PASS — verified on every EMG pulse
P3	Fault implies all-red (fault_out |-> both lights red)	PASS — verified on all fault events

10.3 WAVEFORM — NORMAL OPERATION AND FAULT RESPONSE (0 – 500 ns):

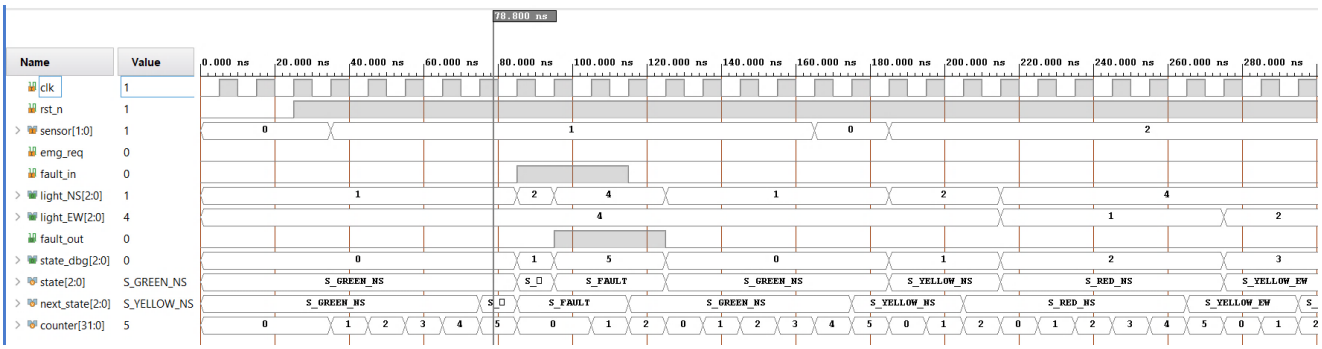
The waveform shows the overall simulation behavior of the Traffic Light Controller, including normal operation and fault injection scenarios. The FSM cycles through the four primary states — S_GREEN_NS, S_YELLOW_NS, S_RED_NS, and S_YELLOW_EW — demonstrating correct phase sequencing.

The light_EW signal exhibits all valid encodings: 1 (Green during S_RED_NS), 2 (Yellow during S_YELLOW_EW), and 4 (Red during NS phases), confirming proper output behavior. The counter resets to zero at every state transition, ensuring precise phase timing and preventing drift across FSM cycles.

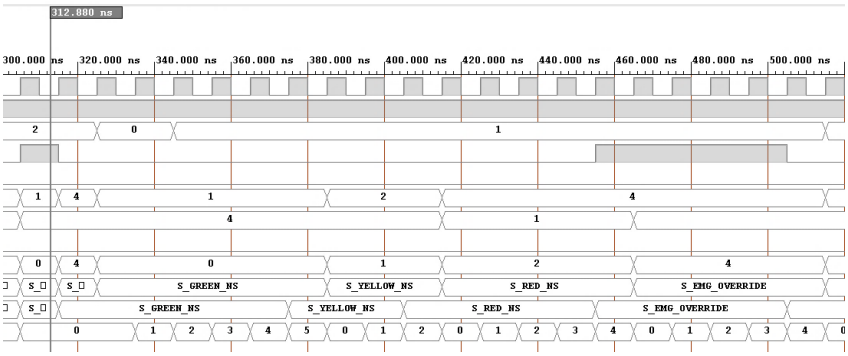
Fault injection events are also observed, where fault_in pulses drive the FSM into S_FAULT (state_dbg = 5), forcing both light_NS and light_EW to red and asserting fault_out. Upon de-assertion of fault_in, the FSM recovers correctly to S_GREEN_NS, satisfying the fault safety property.

This confirms correct FSM sequencing under both nominal and fault conditions.

(0-300ns)

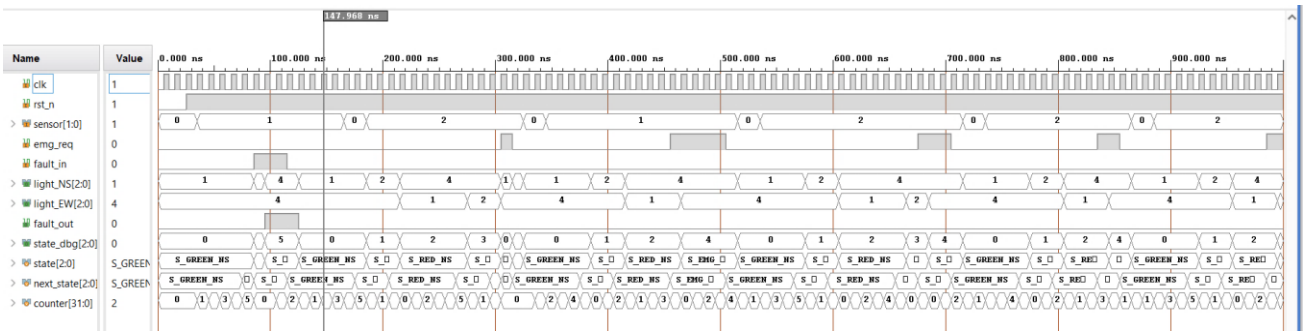


(300-500ns)



10.4 WAVEFORM — EMERGENCY OVERRIDE:

The waveform shows emergency override behavior of the Traffic Light Controller. When emg_req is asserted, the FSM immediately transitions to S_EMG_OVERRIDE (state_dbg = 4) on the next clock edge, forcing both light_NS and light_EW to 4 (Red). After emg_req is de-asserted, the FSM recovers correctly to S_GREEN_NS and resumes normal phase sequencing. Emergency override is observed to occur from multiple FSM states, confirming that the design correctly prioritises emg_req irrespective of the current phase.



11. APPLICATIONS:

Emergency Services Integration

The emg_req interface connects directly to GPS-based transponders fitted to ambulances, fire engines, and police vehicles. When an emergency vehicle approaches the intersection, the transponder asserts emg_req, immediately clearing the intersection to all-red. This reduces emergency response times and minimises the risk of collision at intersections.

Smart City Infrastructure

The sensor[1:0] input is designed to interface with inductive loop or radar detectors. A future RTL extension could use these readings for dynamic green-time adjustment, enabling green-wave coordination across arterial roads.

FPGA Prototyping and ASIC Design

The parameterised, synthesis-safe RTL can be deployed directly on an FPGA development board for hardware-in-the-loop testing. The design also adheres to ASIC coding guidelines — synchronous reset, always_ff/always_comb separation — making it suitable for gate-level synthesis, static timing analysis, and design-for-testability insertion.

12. INFERENCE:

The Traffic Light Controller was successfully verified using a SystemVerilog layered testbench. Key outcomes are summarised below.

- All 19,999 scoreboard checks passed with zero failures across 200 constrained-random transactions. Every 5th transaction forced inject_emg=1 and every 7th forced inject_fault=1, guaranteeing emg_override and fault_state coverage bins were closed despite the probabilistic constraints.
- Functional coverage reached 94.4%, with all six FSM states, all four sensor combinations, EMG trigger, and fault trigger bins closed.
- SVA properties P1, P2, and P3 passed throughout the entire simulation — no conflicting green was ever produced, emergency override forced all-red within exactly one clock cycle, and fault_out always implied all-red outputs.
- The phase counter resets on every state transition, ensuring each state holds for exactly GREEN_TIME or YELLOW_TIME cycles before advancing.
- Simulation termination is controlled from the top-level module initial block, ensuring all forked threads stop cleanly after the scoreboard report is printed

This structured approach confirms that the TLC design is functionally correct across all six states, all priority levels of the arbitration chain, and all corner-case input combinations covered by the verification plan.